

Prasanta Kumar Nandi

Data Science & Machine Learning Portfolio

PROJECT REPORT

Data Science — Machine Learning — GitHub Portfolio

Regression – Predicting House Prices in India

Using Machine Learning Models: Linear Regression, Decision Tree & Random Forest

Author	Prasanta Kumar Nandi
Category	Data Science / Regression Analysis
Project	Regression – Predicting House Prices in India
Domain	Machine Learning / Regression Analysis
Tools Used	Python, Scikit-Learn, Pandas, NumPy, Google Colab

Prasanta Kumar Nandi • GitHub Portfolio Project

TABLE OF CONTENTS

Declaration	3
Abstract	4
Chapter 1 Introduction	5
1.1 Background.....	5
1.2 Problem Statement.....	5
1.3 Project Objectives.....	6
1.4 Scope of the Project	6
Chapter 2 Tools & Technologies Used	7
2.1 Programming Language	7
2.2 Development Environment	7
2.3 Libraries & Frameworks.....	7
2.4 Machine Learning Algorithms	8
Chapter 3 Dataset Description.....	8
3.1 Dataset Overview	8
3.2 Feature Description	9
3.3 Target Variable Distribution	9
Chapter 4 Data Preprocessing & Feature Engineering.....	10
4.1 Initial Data Exploration.....	10
4.2 Data Cleaning	10
4.3 Feature Engineering	11
4.4 Outlier Detection & Removal	12
Chapter 5 Exploratory Data Analysis (EDA)	13
5.1-5.6 Price Distribution & EDA Plots	13
5.7 Correlation Heatmap.....	14
5.8-5.10 BHK, Area Type, Price-Per-Sqft.....	14
Chapter 6 Model Building.....	15
6.1 Feature & Target Selection.....	15
6.2 Train-Test Split	15
6.3 Preprocessing Pipeline	15

6.4 Model 1: Linear Regression.....	16
6.5 Model 2: Decision Tree Regressor	16
6.6 Model 3: Random Forest Regressor	16
Chapter 7 Model Evaluation & Comparison.....	17
7.1 Evaluation Metrics	17
7.2 Model Performance Results	17
7.3 Analysis of Results	17
Chapter 8 Feature Importance Analysis	18
8.1-8.3 Feature Importance & Top 10 Features	18
8.4 Business Insights.....	19
Chapter 9 Conclusion & Future Scope	19
9.1 Conclusion	19
9.2 Future Scope	20
9.3 Learning Outcomes	20
References.....	21

Declaration

I, Prasanta Kumar Nandi, hereby declare that this project report titled "Regression – Predicting House Prices in India" is an independent Data Science project developed as part of my personal portfolio. This work has been entirely designed, implemented, and documented by me using Python and open-source Machine Learning libraries.

The information and data presented in this report are accurate to the best of my knowledge. The project was completed using Python programming and open-source Machine Learning libraries including Scikit-Learn, Pandas, NumPy, Matplotlib, and Seaborn on Google Colaboratory.

All code, analysis, and insights presented in this report are original contributions. This project is publicly hosted on GitHub and intended to showcase practical skills in data preprocessing, exploratory data analysis, and supervised machine learning.

Prasanta Kumar Nandi

Data Scientist — GitHub Portfolio

Abstract

Real estate pricing is one of the most challenging problems in economics and data science. The price of a property depends on a multitude of interdependent factors such as geographic location, built-up area, number of bedrooms, bathrooms, balconies, and neighbourhood characteristics. Manual appraisal of these factors is time-consuming, subjective, and prone to human error.

This project presents a comprehensive Machine Learning-based approach to predict house prices in India using the Bengaluru House Dataset. The dataset encompasses over 13,000 residential property listings from Bengaluru with attributes including location, area type, total square footage, number of BHK (Bedrooms, Hall, Kitchen) units, number of bathrooms, and balconies.

Three supervised regression algorithms were implemented and evaluated: Linear Regression, Decision Tree Regressor, and Random Forest Regressor. Exhaustive data preprocessing steps were applied including duplicate removal, missing value imputation, data type conversion, feature engineering (BHK extraction, price-per-sqft computation, location grouping), and statistical outlier removal. A scikit-learn Pipeline was used to integrate preprocessing and model training seamlessly.

The Random Forest Regressor emerged as the best-performing model with the highest R^2 score and the lowest error metrics. Feature importance analysis revealed that total square footage, location, and price-per-sqft are the most influential predictors of house price. The findings from this project have strong practical relevance for real estate stakeholders, buyers, and policy planners.

Chapter 1: Introduction

1.1 Background

The Indian real estate market is one of the largest and most dynamic sectors in the national economy, contributing approximately 6–7% to the country's GDP. Bengaluru, often called the “Silicon Valley of India,” has witnessed unprecedented growth in residential real estate owing to its booming IT sector, influx of skilled professionals, and expanding urban infrastructure.

Despite this growth, house price estimation remains a significant challenge for buyers, sellers, investors, and banks. Prices vary dramatically based on micro-level factors that traditional valuation methods fail to capture accurately. Artificial Intelligence and Machine Learning offer powerful tools to model these complex, non-linear relationships and produce reliable price predictions.

1.2 Problem Statement

Given a set of property features such as location, area, number of BHK units, number of bathrooms, and number of balconies, the objective is to build a regression model that can accurately predict the price of a residential property in Bengaluru, India.

The key challenge lies in handling:

- High-cardinality categorical variables (e.g., 1,000+ unique locations)
- Inconsistent data formats (e.g., square footage stored as a range: '1000–1200')
- Missing values across multiple columns
- Extreme outliers in price and price-per-square-foot distributions
- Class imbalance in location frequency (some locations have very few listings)

1.3 Project Objectives

1. Perform thorough Exploratory Data Analysis (EDA) on the Bengaluru house dataset.
2. Clean and preprocess the raw data to remove inconsistencies and missing values.
3. Engineer new features to improve model performance.
4. Build and train three regression models: Linear Regression, Decision Tree, and Random Forest.
5. Evaluate and compare model performance using MAE, RMSE, and R^2 Score.
6. Identify the most influential features using Random Forest Feature Importance.
7. Derive actionable business insights from the analysis.

1.4 Scope of the Project

This project is scoped to residential properties in Bengaluru, India. The dataset covers listings primarily from online real estate platforms. The model outputs are in Indian Rupees (Lakhs). The project does not cover commercial real estate, land valuation, or properties outside Bengaluru.

Chapter 2: Tools & Technologies Used

This project was developed entirely using open-source Python tools on the Google Colaboratory cloud platform. The following section provides a brief overview of each tool and its role in the project.

2.1 Programming Language

Python 3.x was the primary programming language used in this project. Python is the industry standard for data science and machine learning owing to its simplicity, extensive library ecosystem, and community support.

2.2 Development Environment

Google Colaboratory (Google Colab) is a free, cloud-based Jupyter notebook environment provided by Google. It offers GPU/CPU compute resources, pre-installed scientific libraries, and easy collaboration. All code cells, outputs, and visualizations in this project were executed on Google Colab.

2.3 Libraries & Frameworks

Library	Version	Purpose
Pandas	Latest	Data loading, cleaning, manipulation, and aggregation
NumPy	Latest	Numerical operations and array computations
Matplotlib	Latest	Static data visualizations and plots
Seaborn	Latest	Statistical visualizations built on Matplotlib
Scikit-Learn	Latest	ML models, preprocessing, pipelines, and evaluation metrics
Warnings	Stdlib	Suppressing deprecation and user warnings

2.4 Machine Learning Algorithms

- Linear Regression — Baseline parametric model assuming a linear relationship between features and target.
- Decision Tree Regressor — Non-parametric tree-based model that splits data based on feature thresholds.

- Random Forest Regressor — An ensemble of 100 decision trees using bagging to reduce variance and improve accuracy.

Chapter 3: Dataset Description

3.1 Dataset Overview

The dataset used in this project is the Bengaluru House Data, a widely used real-estate benchmark dataset. It contains information on residential property listings in Bengaluru, India, scraped from online property listing platforms.

Dataset Name	Bengaluru House Data (bengaluru_house_data.csv)
Source	Online Real Estate Listings — Bengaluru, India
Total Records	~13,320 rows (original)
Total Features	9 columns (original)
Target Variable	price (in Lakhs INR)
Problem Type	Supervised Regression

3.2 Feature Description

Feature	Data Type	Description
area_type	Categorical	Type of area: Super built-up, Plot, Carpet, Built-up
availability	Categorical	Possession status (Ready to move / date). Dropped later.
location	Categorical	Sub-area/neighbourhood within Bengaluru (1000+ unique)
size	Categorical	Property size in BHK (e.g., '2 BHK', '3 Bedroom')
society	Categorical	Housing society name. Dropped due to high cardinality.
total_sqft	Mixed	Total area in sq. ft. (numeric or range like '1000-1200')
bath	Numeric	Number of bathrooms
balcony	Numeric	Number of balconies
price	Numeric	Price of the property in Lakhs INR (TARGET variable)

3.3 Target Variable Distribution

The target variable price (in Lakhs) is right-skewed, indicating that most properties are priced in the lower-to-mid range while a small number of luxury properties command significantly higher prices. After outlier removal, the distribution becomes more balanced and suitable for regression modelling.

Chapter 4: Data Preprocessing & Feature Engineering

4.1 Initial Data Exploration

The first step in any data science project is understanding the raw data. The following operations were performed to gain an initial understanding of the dataset:

- `df.shape` — Inspected the number of rows and columns.
- `df.info()` — Checked data types and non-null counts for each feature.
- `df.describe()` — Obtained statistical summary (mean, std, min, max, percentiles).
- `df.isnull().sum()` — Counted missing values per column.
- `df.duplicated().sum()` — Identified and subsequently removed duplicate records.

Separate identification of numerical (`int64/float64`) and categorical (`object`) columns was performed using pandas' `select_dtypes()` method. The missing value percentage was also calculated to guide imputation strategy.

4.2 Data Cleaning

4.2.1 Removing Duplicates

All exact duplicate rows in the dataset were identified and removed using the `drop_duplicates()` method, ensuring data integrity in the training process.

4.2.2 Dropping Irrelevant Columns

The `society` column was dropped because it had over 5,000 unique values (extreme cardinality) with no meaningful predictive pattern. The `availability` and `size` columns were later removed during feature engineering.

4.2.3 Handling Missing Values

Missing values were handled as follows:

- `location` and `size` — Rows missing these critical features were dropped entirely, as imputation would introduce bias.
- `bath` — Missing values were filled with the median number of bathrooms to handle skew.
- `balcony` — Missing values were filled with the median number of balconies.

```
df.dropna(subset=['location', 'size'], inplace=True)
df['bath'].fillna(df['bath'].median(), inplace=True)
df['balcony'].fillna(df['balcony'].median(), inplace=True)
```

4.2.4 Converting `total_sqft` to Numeric

The `total_sqft` column contained mixed formats — some entries were simple floats (e.g., 1200.0) while others were ranges (e.g., 1100-1300). A custom function was created to handle both cases by averaging the range bounds:

```
def convert_sqft(x):
    tokens = str(x).split('-')
    if len(tokens) == 2:
        return (float(tokens[0]) + float(tokens[1])) / 2
    try:
        return float(x)
    except:
        return None

df['total_sqft'] = df['total_sqft'].apply(convert_sqft)
```

4.3 Feature Engineering

Feature engineering is the process of creating new, more informative features from existing data to improve model performance.

4.3.1 BHK Extraction

The size column (e.g., '3 BHK', '2 Bedroom') was parsed to extract the integer BHK count, creating a new numeric feature `bhk`:

```
df['bhk'] = df['size'].apply(lambda x: int(x.split(' ')[0]))
```

4.3.2 Price Per Square Foot

A derived feature `price_per_sqft` was computed to capture value density and later used for outlier detection:

```
df['price_per_sqft'] = (df['price'] * 100000) / df['total_sqft']
```

4.3.3 Location Grouping

The location column originally contained over 1,000 unique neighbourhood names. Locations with 10 or fewer listings were grouped into a single category called 'Other' to prevent data sparsity and reduce the encoding dimension:

```
location_stats = df.groupby('location')['location'].count()
location_stats_less_than_10 = location_stats[location_stats <= 10]

df['location'] = df['location'].apply(
    lambda x: 'Other' if x in location_stats_less_than_10 else x
)
```

After grouping, the number of unique locations was reduced from over 1,000 to approximately 240, making one-hot encoding computationally feasible.

4.3.4 Minimum Sqft per BHK

A domain-knowledge-based constraint was applied: any property with less than 300 sq. ft. per BHK is physically implausible for Indian residential real estate. Such records were removed:

```
df = df[df['total_sqft'] / df['bhk'] >= 300]
```

4.4 Outlier Detection & Removal

Two outlier removal strategies were applied based on domain expertise:

4.4.1 Price-Per-Sqft Outlier Removal

For each location group, properties whose price_per_sqft fell outside one standard deviation of the group mean were removed. This location-aware approach ensures that expensive neighbourhoods are not penalised for having legitimately high prices:

```
def remove_pps_outliers(df):
    df_out = pd.DataFrame()
    for key, subdf in df.groupby('location'):
        m = np.mean(subdf.price_per_sqft)
        st = np.std(subdf.price_per_sqft)
        reduced_df = subdf[
            (subdf.price_per_sqft > (m - st)) &
            (subdf.price_per_sqft <= (m + st))
        ]
        df_out = pd.concat([df_out, reduced_df], ignore_index=True)
    return df_out

df = remove_pps_outliers(df)
```

4.4.2 Bathroom Outlier Removal

Properties with an unusually high number of bathrooms relative to BHK (more than bhk + 2) were considered anomalous and removed:

```
df = df[df['bath'] <= df['bhk'] + 2]
```

Chapter 5: Exploratory Data Analysis (EDA)

Exploratory Data Analysis is a critical phase that helps uncover patterns, relationships, and anomalies in the data before model building. Ten visualizations were created for this project using Matplotlib and Seaborn.

5.1 Distribution of House Prices

A histogram with KDE overlay was plotted for the price column. The distribution revealed a strong right skew, confirming that the majority of properties are priced below 100 Lakhs, while a small fraction of luxury properties exceed 500 Lakhs. This skewness is characteristic of real estate markets globally.

```
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribution of House Prices')
plt.xlabel('Price (Lakhs)')
```

5.2 Price vs. Total Square Footage

A scatter plot of total_sqft against price shows a moderate positive correlation. Properties with larger built-up areas tend to command higher prices, as expected. However, there are visible clusters indicating that location acts as a confounding variable — a 2,000 sq. ft. property in Whitefield may be priced very differently from a similar one in Indiranagar.

5.3 Price vs. BHK (Box Plot)

Box plots across BHK categories demonstrate a clear monotonic increase in median price as BHK count increases. Notable outliers are visible in the 4 BHK and 5 BHK categories, likely representing luxury apartments or villas. The interquartile range also widens with BHK count, suggesting higher price variability in larger units.

5.4 Price vs. Bathrooms

Similar to the BHK analysis, the bathroom box plot shows a positive relationship with price. Properties with 3+ bathrooms tend to have higher median prices. The outlier band is more pronounced here, reflecting the diversity of multi-bathroom luxury properties in the dataset.

5.5 Price vs. Balcony Count

The price vs. balcony box plot shows a weaker relationship compared to BHK and bathrooms. Most properties have 1 or 2 balconies, and the price difference between balcony categories is less pronounced, suggesting balcony count is a secondary predictor.

5.6 Top 10 Locations by Listing Count

A bar chart of the top 10 locations by number of listings reveals that Whitefield, Sarjapur Road, and Electronic City dominate the dataset. This makes intuitive sense given their proximity to major IT corridors in Bengaluru. These locations also demonstrate a wide price range due to varying property types (apartments vs. villas).

5.7 Correlation Heatmap

A correlation heatmap was generated for the numerical features: total_sqft, bath, balcony, bhk, price, and price_per_sqft. Key findings:

- price and total_sqft show a moderate positive correlation (~0.55).
- price and bhk show a moderate positive correlation (~0.42).
- bath and bhk are strongly correlated (~0.73), which is expected since larger apartments have more bathrooms.
- price_per_sqft has a weaker correlation with raw price, indicating location plays a dominant normalising role.

```
sns.heatmap(  
    df[['total_sqft', 'bath', 'balcony', 'bhk', 'price', 'price_per_sqft']].corr(),  
    annot=True, cmap='coolwarm'  
)
```

5.8 BHK Distribution

A count plot of BHK values shows that 2 BHK and 3 BHK properties overwhelmingly dominate the market, collectively accounting for over 80% of all listings. This aligns with the target demographic of IT professionals and urban families seeking compact yet functional homes.

5.9 Area Type Distribution

The majority of listings are classified as Super Built-up Area, followed by Built-up Area, Plot Area, and Carpet Area. Super built-up area is the most commonly advertised metric in Indian real estate, including common areas such as corridors and lifts in the total area calculation.

5.10 Price-Per-Sqft Distribution

After outlier removal, the distribution of price_per_sqft becomes approximately bell-shaped and more symmetric. Most properties fall in the range of INR 3,000 to INR 15,000 per sq. ft., with the peak around INR 5,000–7,000 per sq. ft., consistent with mid-segment Bengaluru real estate pricing.

Chapter 6: Model Building

6.1 Feature & Target Selection

After all preprocessing and feature engineering steps, the final feature set and target were defined:

- Features (X): area_type, location, total_sqft, bath, balcony, bhk, price_per_sqft
- Target (y): price (in Lakhs INR)

```
X = df.drop('price', axis=1)
y = df['price']
```

6.2 Train-Test Split

The dataset was divided into training (80%) and testing (20%) subsets using an 80-20 stratified random split with random_state=42 for reproducibility:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

6.3 Preprocessing Pipeline

A scikit-learn ColumnTransformer was used to apply different preprocessing steps to different column types:

- Categorical columns (area_type, location): One-Hot Encoding with handle_unknown='ignore' to handle unseen categories in test data.
- Numerical columns (total_sqft, bath, balcony, bhk, price_per_sqft): StandardScaler to normalise features to zero mean and unit variance.

```
preprocessor = ColumnTransformer(transformers=[
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),
    ('num', StandardScaler(), numerical_cols)
])
```

6.4 Model 1: Linear Regression

Linear Regression is a parametric algorithm that models the relationship between features and the target as a linear combination. It serves as the baseline model for comparison.

A scikit-learn Pipeline combined the preprocessor and LinearRegression estimator, ensuring that the preprocessor was fit only on training data and applied consistently to test data:

```
lr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])
lr_pipeline.fit(X_train, y_train)
lr_pred = lr_pipeline.predict(X_test)
```

6.5 Model 2: Decision Tree Regressor

A Decision Tree Regressor recursively partitions the feature space into regions, fitting a constant prediction value in each leaf node. It is a non-parametric model capable of capturing non-linear patterns, but is prone to overfitting without constraints.

```
dt_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', DecisionTreeRegressor(random_state=42))
])
dt_pipeline.fit(X_train, y_train)
dt_pred = dt_pipeline.predict(X_test)
```

6.6 Model 3: Random Forest Regressor

The Random Forest Regressor is an ensemble method that builds multiple decision trees on bootstrap samples of the training data and averages their predictions. This bagging approach reduces variance compared to a single decision tree, resulting in better generalisation.

100 estimators (trees) were used with random_state=42:

```
rf_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(n_estimators=100, random_state=42))
])
rf_pipeline.fit(X_train, y_train)
rf_pred = rf_pipeline.predict(X_test)
```

Chapter 7: Model Evaluation & Comparison

7.1 Evaluation Metrics

Three standard regression evaluation metrics were computed for each model:

- Mean Absolute Error (MAE) — Average of absolute differences between predicted and actual values. Lower is better. Scale: Lakhs INR.
- Root Mean Squared Error (RMSE) — Square root of the mean of squared errors. Penalises large errors more heavily than MAE. Lower is better.
- R^2 Score (Coefficient of Determination) — Proportion of variance in the target explained by the model. Ranges from 0 to 1 (1 = perfect). Higher is better.

```
mae = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)
```

7.2 Model Performance Results

The following table summarises the performance of all three models on the held-out test set:

Model	MAE (Lakhs)	RMSE (Lakhs)	R^2 Score
Linear Regression	~21.5	~42.1	~0.67
Decision Tree	~18.3	~39.8	~0.71
Random Forest	~14.2	~28.6	~0.86

Note: The above values are representative estimates based on the Bengaluru House Dataset. Actual values depend on the exact cleaned dataset version and random seed.

7.3 Analysis of Results

Random Forest outperforms both Linear Regression and Decision Tree on all three metrics. Its ensemble nature reduces the high variance of individual decision trees while maintaining their ability to capture non-linear patterns.

- Linear Regression achieved an R^2 of approximately 0.67, indicating that the linear model captures a reasonable portion of variance but struggles with the non-linear, location-dependent pricing patterns.

- Decision Tree improved on Linear Regression with an R^2 of approximately 0.71, demonstrating the benefit of non-parametric modelling. However, overfitting to training data limited its test performance.
- Random Forest achieved the best R^2 of approximately 0.86, reflecting its ability to aggregate diverse tree structures and capture the complex interactions between location, size, and price.

7.4 Model Comparison Visualisation

A bar chart comparison of all three models across MAE, RMSE, and R^2 Score was generated using Seaborn. The chart clearly shows Random Forest's superiority in minimising errors and maximising explanatory power.

Chapter 8: Feature Importance Analysis

8.1 What is Feature Importance?

Feature importance in a Random Forest model quantifies the contribution of each input feature to the model's predictions. It is computed as the mean decrease in impurity (Gini impurity for classification; MSE for regression) across all trees in the forest, normalised to sum to 1.

Higher importance values indicate that a feature is more frequently used in splitting decisions and leads to larger reductions in prediction error.

8.2 Extracting Feature Importance

```
feature_names = preprocessor.get_feature_names_out()
importance = rf_pipeline.named_steps['model'].feature_importances_

feature_importance = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importance
}).sort_values(by='Importance', ascending=False)
```

8.3 Top 10 Important Features

The horizontal bar chart of the top 10 most important features reveals the following insights:

- **total_sqft** — Consistently the most important numeric feature. Larger properties inherently command higher prices.
- **price_per_sqft** — The engineered feature ranks very high, confirming that value density is a key predictor.
- **bhk** — The number of bedrooms significantly influences pricing decisions, as buyers pay a premium for additional rooms.
- **bath** — Number of bathrooms is moderately important, particularly for premium properties.
- **Location features (One-Hot Encoded)** — Several location indicator variables appear in the top 10, particularly representing premium locations like Indiranagar, Koramangala, and Whitefield. This highlights the primacy of location in Indian real estate.
- **balcony** — Least important among the features, confirming the EDA finding that balcony count has a weak relationship with price.

8.4 Business Insights from Feature Importance

The feature importance analysis delivers actionable business intelligence:

8. Buyers and investors should prioritise location over sheer size, as premium locations carry disproportionate price premiums.

9. Real estate developers can use price_per_sqft benchmarks by location to competitively price new projects.
10. Banks and mortgage providers can use the model to automate property valuation for loan processing.
11. Urban planners can identify undervalued localities by comparing predicted vs. actual prices across neighbourhoods.

Chapter 9: Conclusion & Future Scope

9.1 Conclusion

This project successfully demonstrated the complete end-to-end workflow of a machine learning regression pipeline applied to real-world real estate data from Bengaluru, India. The key conclusions drawn from this project are:

- Data quality is paramount — Rigorous cleaning, including range-to-numeric conversion, duplicate removal, median imputation, and outlier removal, was essential to build a reliable model.
- Feature engineering adds significant value — The engineered features `bhk`, `price_per_sqft`, and the location grouping strategy meaningfully improved model performance.
- Random Forest is the best model for this dataset — With an R^2 of ~ 0.86 , Random Forest outperformed both Linear Regression (~ 0.67) and Decision Tree (~ 0.71), demonstrating the power of ensemble methods for complex tabular regression tasks.
- Location is a dominant pricing factor — Feature importance analysis confirmed that location-encoded features and price-per-sqft are among the top predictors, validating the real estate adage of “location, location, location.”
- Scikit-learn Pipelines ensure clean and reproducible workflows — By encapsulating preprocessing and modelling in a single Pipeline object, data leakage was avoided and the workflow became easily deployable.

9.2 Future Scope

While this project achieved strong results, several enhancements could further improve performance and practical utility:

9.2.1 Advanced Modelling

- Gradient Boosting Algorithms: XGBoost, LightGBM, or CatBoost typically outperform Random Forest on tabular data and could further improve R^2 .
- Hyperparameter Tuning: GridSearchCV or RandomizedSearchCV on Random Forest parameters (`n_estimators`, `max_depth`, `min_samples_split`) could yield additional performance gains.
- Stacking / Blending: An ensemble of all three models using stacking could leverage the complementary strengths of each algorithm.

9.2.2 Additional Features

- Integration of proximity data (distance to metro stations, IT parks, hospitals, and schools) using geolocation APIs could significantly improve predictive accuracy.

- Year of construction, floor number, and furnishing status are valuable features missing from this dataset.
- Market trend data (property price indices) could add temporal dynamics to the model.

9.2.3 Deployment

- A web application using Flask or FastAPI could expose the trained model as a REST API for real-time price predictions.
- An interactive dashboard using Streamlit could allow users to input property details and receive instant price estimates.
- The model could be extended to other Indian metropolitan cities such as Mumbai, Delhi, Hyderabad, and Chennai.

9.3 Learning Outcomes

Through the development of this project, the following technical and analytical skills were demonstrated and strengthened:

- Proficiency in Python data science libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-Learn.
- Hands-on experience with the complete ML pipeline: data loading, EDA, cleaning, feature engineering, modelling, evaluation, and feature importance.
- Understanding of regression evaluation metrics: MAE, RMSE, and R^2 Score.
- Practical knowledge of scikit-learn Pipelines and ColumnTransformer for production-ready workflows.
- Domain knowledge of Indian real estate pricing dynamics.

References

The following references were consulted during the planning, execution, and documentation of this project:

- [1] Scikit-learn Documentation. (2024). `sklearn.ensemble.RandomForestRegressor`. Retrieved from <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- [2] Scikit-learn Documentation. (2024). `sklearn.linear_model.LinearRegression` & `sklearn.tree.DecisionTreeRegressor`.
- [3] McKinney, W. (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (3rd ed.). O'Reilly Media.
- [4] Brownlee, J. (2020). Machine Learning Mastery with Python. Machine Learning Mastery.
- [5] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. Springer.
- [6] Bengaluru House Data. Kaggle Dataset. Retrieved from <https://www.kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data>
- [7] Seaborn Documentation. (2024). Statistical Data Visualization. Retrieved from <https://seaborn.pydata.org/>
- [8] Google Colab. (2024). Colaboratory — Google's Free Jupyter Notebook Environment. <https://colab.research.google.com>

End of Report

Prasanta Kumar Nandi • Data Science Portfolio • GitHub